

Winter 2026

ENERGY295-Electrochemical Energy Storage Systems: Modeling and Estimation

Lecture 13

SIMONA ONORI, Stanford University

Energy Science and Engineering

Electrical Engineering (courtesy)

Precourt Institute of Energy



solving energy
for humanity.

Stanford
School of Sustainability
Energy Science & Engineering

Recap

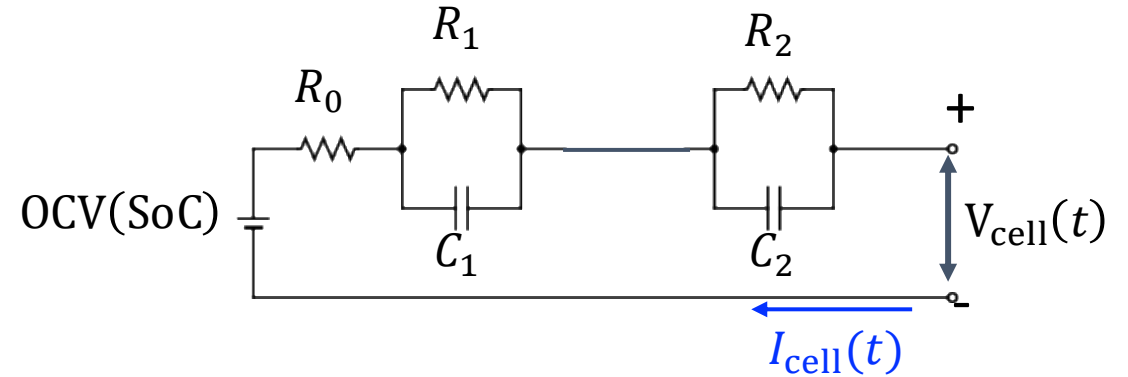
- LTI: continuous and discrete time
- Luenberger's Observer (deterministic, pole placement)
- Observability
- Condition Number $\rightarrow$ decision making metric for sensor selection

2nd order Equivalent Circuit Model (ECM)

State equations

$$\begin{aligned}\dot{SOC} &= -\frac{1}{Q}I \\ \dot{V}_1 &= -\frac{1}{R_1 C_1} V_1 + \frac{1}{C_1} I \\ \dot{V}_2 &= -\frac{1}{R_2 C_2} V_2 + \frac{1}{C_2} I\end{aligned}$$

$$x = \begin{bmatrix} SOC \\ V_1 \\ V_2 \end{bmatrix}, u = I$$



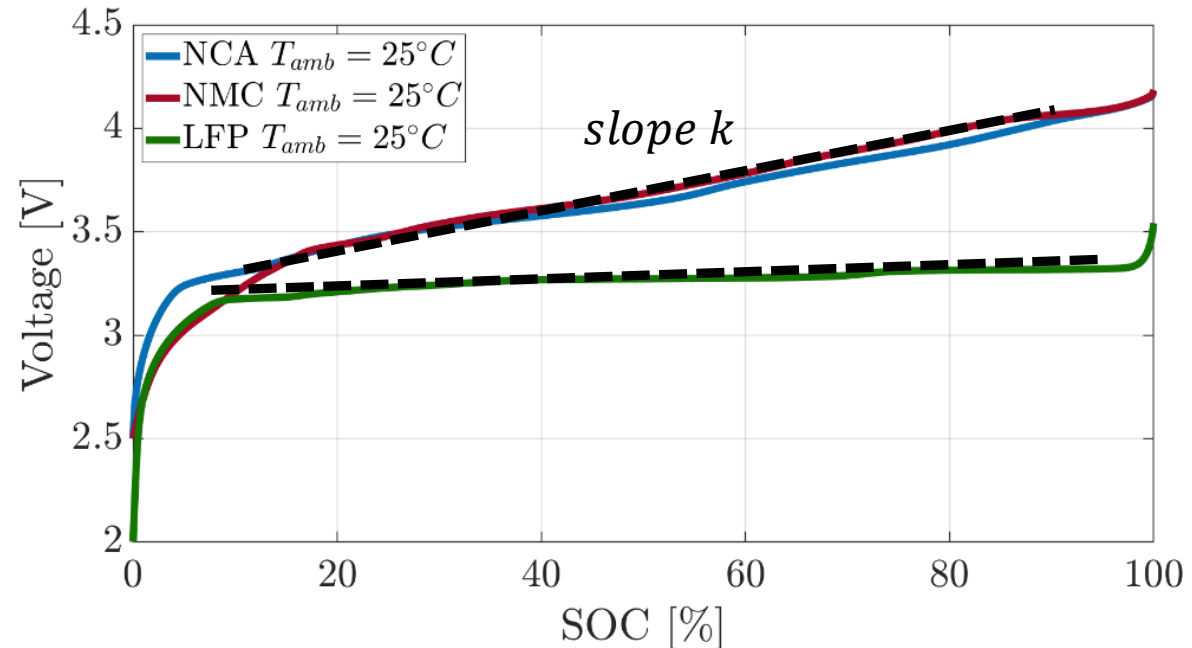
Output (KVL)

$$V = OCV(SOC) - R_0 I - V_1 - V_2$$

Let

$$k = \left. \frac{dOCV}{dSOC} \right|_{SOC_0}$$

$$OCV = k SOC$$



2nd order ECM

(linearized) State Space

$$A = \begin{bmatrix} 0 & 0 & 0 \\ 0 & -\frac{1}{R_1 C_1} & 0 \\ 0 & 0 & -\frac{1}{R_2 C_2} \end{bmatrix}, \quad B = \begin{bmatrix} -\frac{1}{Q} \\ \frac{1}{C_1} \\ \frac{1}{C_2} \end{bmatrix}$$

$$C = [k \quad -1 \quad -1], \quad D = [-R_0]$$

2nd order ECM - Observability

For a 3-state LTI (linearized) system

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \end{bmatrix}$$

$$CA = \begin{bmatrix} 0 & \frac{1}{R_1 C_1} & \frac{1}{R_2 C_2} \end{bmatrix},$$
$$CA^2 = \begin{bmatrix} 0 & -\left(\frac{1}{R_1 C_1}\right)^2 & -\left(\frac{1}{R_2 C_2}\right)^2 \end{bmatrix}$$

$$\mathcal{O} = \begin{bmatrix} k & \frac{-1}{R_1 C_1} & \frac{-1}{R_2 C_2} \\ 0 & \frac{1}{R_1 C_1} & \frac{1}{R_2 C_2} \\ 0 & -\left(\frac{1}{R_1 C_1}\right)^2 & -\left(\frac{1}{R_2 C_2}\right)^2 \end{bmatrix}$$

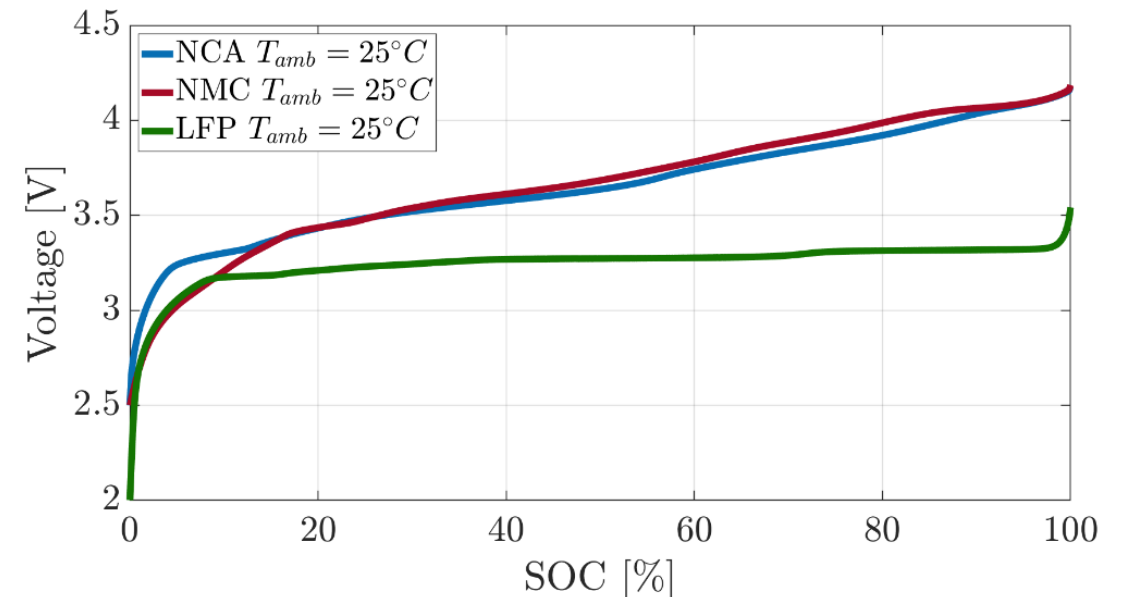
Rank of $\mathcal{O}$

The rank is **3 (full rank)** if and only if:

$$k \neq 0 \text{ and } a_1 \neq a_2$$

If $k = \frac{dOCV}{dSOC}$ is very small but not zero, then:

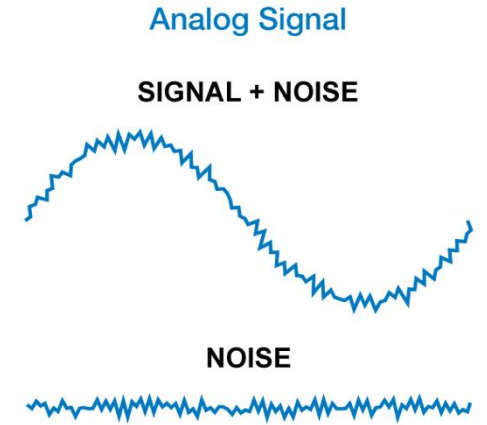
- $\mathcal{O}$ is full rank (rank 3) in theory
- the smallest singular value $\sigma_{min}(\mathcal{O})$ becomes very small, so $\kappa(\mathcal{O}) = \frac{\sigma_{max}}{\sigma_{min}}$ becomes very large $\rightarrow$ high noise amplification in state reconstruction
- $k \approx 0$ Voltage gives you almost no SOC information, any attempt to infer SOC from V becomes extremely noise-sensitive.
 $\Rightarrow$ SOC is observable but extremely ill-conditioned $\Rightarrow$ SOC estimate is fragile / noise-amplifying, even if V_1, V_2 are still well estimated.



Kalman Filter

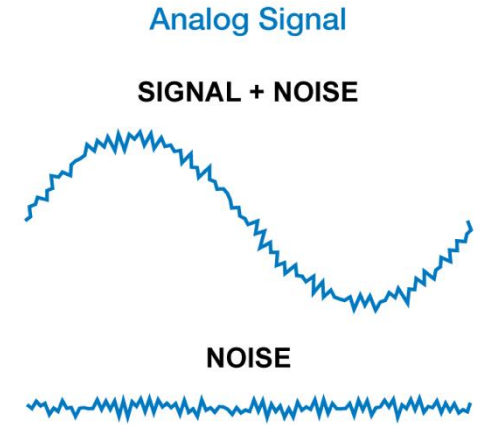
Kalman filter: motivation

- Luenberger's observer is designed for deterministic systems, where system dynamics and measurements are assumed to be **noise-free** or **where disturbances are not modeled probabilistically** (feedback from $(y_{mea} - \hat{y})$).



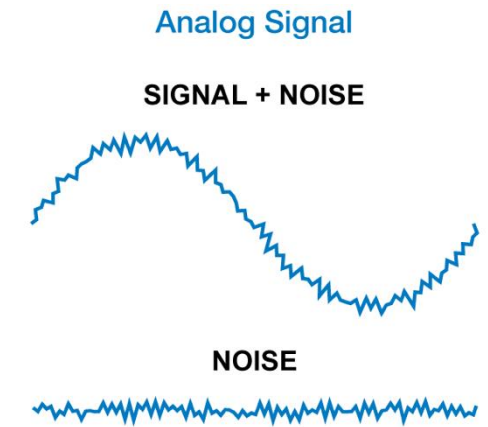
Kalman filter: motivation

- Luenberger's observer is designed for deterministic systems, where system dynamics and measurements are assumed to be **noise-free** or **where disturbances are not modeled probabilistically** (feedback from $(y_{mea} - \hat{y})$).
- In practical systems, sensor measurements are corrupted by noise, and state transitions are also affected by disturbances arising from stochastic processes.



Kalman filter: motivation

- Luenberger's observer is designed for deterministic systems, where system dynamics and measurements are assumed to be **noise-free** or **where disturbances are not modeled probabilistically** (feedback from $(y_{mea} - \hat{y})$).
- In practical systems, sensor measurements are corrupted by noise, and state transitions are also affected by disturbances arising from stochastic processes.
- When the statistical properties of these noises are known or can be modeled, state estimation can be performed more effectively than simply treating sensor signals as noise-free and relying on a deterministic state-transition model.



Kalman Filter

- The Kalman filter (1960) is an **optimal, recursive** state estimator based on a state-space representation of the system.
- It infers unknown states from **noisy, and uncertain process and measurements**.
- When the process and measurement noise are Gaussian, the Kalman filter **minimizes the mean-square estimation error**.
- The filter is **recursive**, meaning that new measurements are incorporated sequentially as they arrive, and the **Kalman gain** is updated over time.
- Importantly, the Kalman filter does not simply filter noisy measurements; it combines measurements with a dynamic model to estimate the system state
- Uncertainty Quantification in Data-Centric Simulations ENERGY 160, (Spring)



Kalman Filter: ingredients

In order to design a Kalman filter we need:

- Series of measurements of the system to be estimated
- Knowledge of the mathematical linear model which describes the system
- Statistical model of the measurement and process noise

System model: Discrete State Space

- State-space model (Luenberger Observer)

$$\begin{aligned}x_{k+1} &= Ax_k + Bu_k \\ y_k &= Cx_k\end{aligned}$$

- State-space model (Kalman Filter)

$$\begin{aligned}x_{k+1} &= Ax_k + Bu_k + w_k \\ y_k &= Cx_k + v_k\end{aligned}$$

Both the **measurements** and the **state-transition model** are assumed to be affected by noise:

- Process noise models uncertainty or disturbances in the system dynamics.
- Measurement noise models sensor imperfections.

Disturbances affecting the system are modeled as stochastic processes.

System model: discrete time state space description

- Discrete time linear system with additive white noise that models unpredictable disturbances.

$$\begin{aligned}x(k+1) &= Ax(k) + Bu(k) + w(k) \\ y(k) &= Cx(k) + v(k)\end{aligned}$$

or

$$\begin{aligned}x_{k+1} &= Ax_k + Bu_k + w_k \\ y_k &= Cx_k + v_k\end{aligned}$$

$x(k)$ or $x_k \in \mathbb{R}^n$	State vector of the discrete-time system
$u(k)$ or $u_k \in \mathbb{R}$	Control input
$w(k)$ or $w_k \in \mathbb{R}^n$	process noise, zero mean, white, Gaussian process noise with covariance $\mathbf{Q}$, $w_k \sim \mathcal{N}(0, \mathbf{Q})$
$y(k)$ or $y_k \in \mathbb{R}$	Observation at time k
$v(k)$ or $v_k \in \mathbb{R}$	observation noise, zero mean Gaussian white noise with covariance $\mathbf{R}$, $v_k \sim \mathcal{N}(0, \mathbf{R})$

Mean, Variance, and Covariance of Random Variables

A random variable is a function that assigns a numerical value to each possible outcome of a random experiment.

Mean (Expected Value)

- The **mean** of a random variable is the average value it takes when the experiment is repeated many times:

$$\mu = E[X]$$

where $E[\cdot]$ denotes the expectation operator. The mean is the expected or average value of the random variable.

Variance

- The variance measures how spread out the values of a random variable are around its mean.

$$\text{Var}(X) = E[(X - E[X])^2]$$

The variance is the expected squared deviation of a random variable from its mean.

Covariance

- The **covariance** between two random variables X and Y measures **how the two variables vary together**.

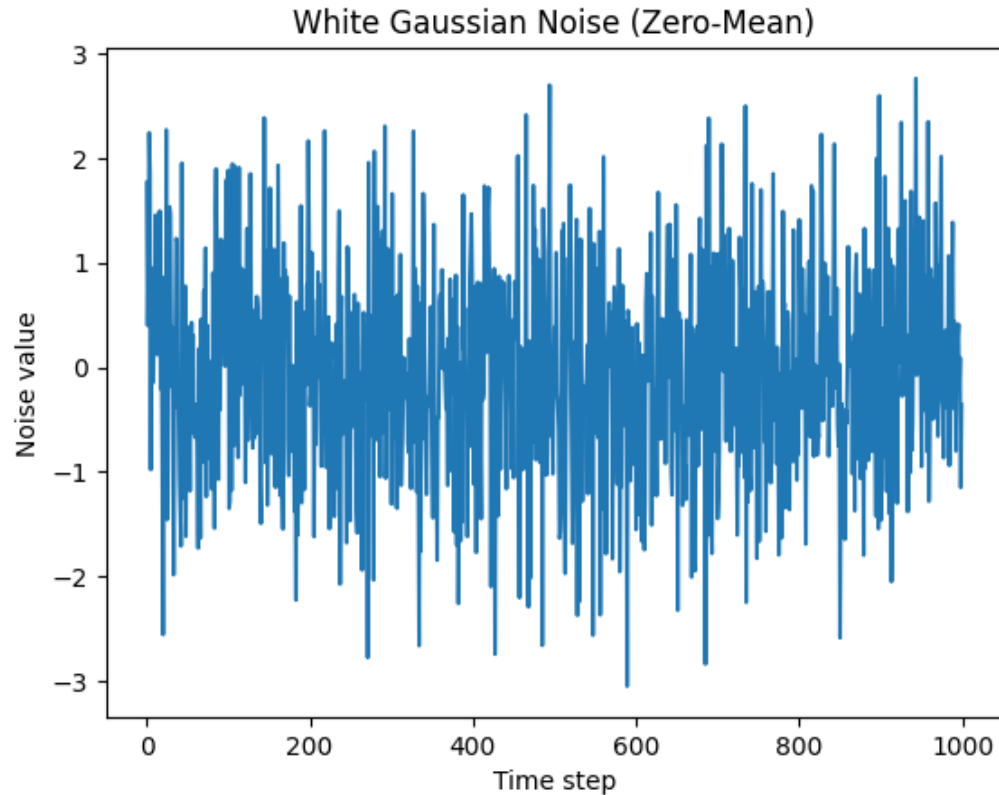
$$\text{Cov}(X, Y) = E[(X - E[X])(Y - E[Y])]$$

Covariance measures the tendency of two random variables to deviate from their means (it can be ≥ 0).

Assumptions

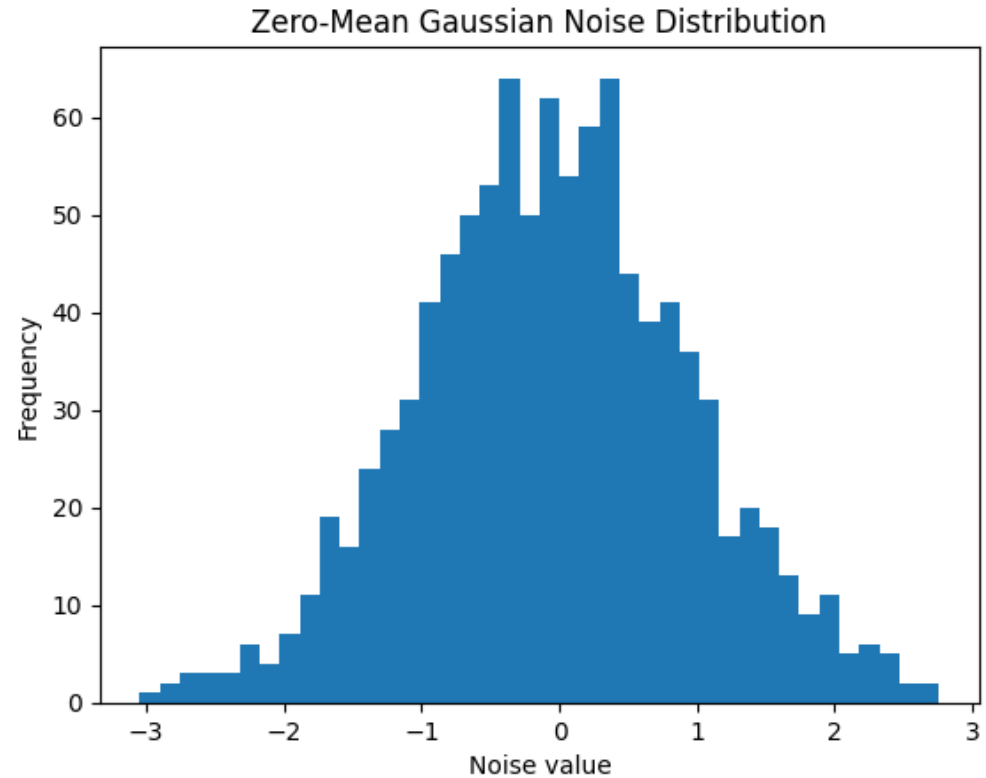
- Process noise and Measurement noise are typically assumed to be zero-mean, Gaussian, and white noise, with known covariance statistics.
- Mathematically, we will model our dynamical system using stochastic differential equations.

White Gaussian Noise (Zero-Mean)



- Random fluctuations over time
- No correlation between samples (→ white)
- Values fluctuate around zero

Zero-Mean Gaussian Noise Distribution



- Zero-Mean Gaussian Noise (distribution)
- Histogram shows the Gaussian (normal) bell shape
- Centered at mean = 0
- Symmetric positive and negative deviations

Scalar Zero-Mean White Gaussian Noise

Let

$$w_k \in \mathbb{R}$$

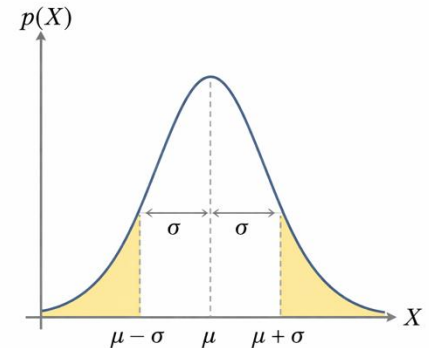
- be a scalar random sequence indexed by discrete time $k = 0, 1, 2, \dots$
- The sequence w_k is **scalar zero-mean white Gaussian noise** if:

1. Zero Mean: $E[w_k] = 0 \rightarrow$ No bias at any time step.
2. Gaussian Distribution (PDF) (**Gaussian** $\rightarrow$ uncertainty follows a bell-shaped distribution)

Each sample w_k is normally distributed: $p(w_k) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{w_k^2}{2\sigma^2}\right)$ or equivalently

3. White (No Temporal Correlation) $E[w_k w_j] = \begin{cases} \sigma^2, & k = j \\ 0, & k \neq j \end{cases}$

$$w_k \sim \mathcal{N}(0, \sigma^2)$$



Vector Zero-Mean White Gaussian Noise (1 / 3)

Let

$$w_k \in \mathbb{R}^n$$

be a random vector sequence (components are random variables). Example (battery ECM): $w_k = \begin{bmatrix} w_{SOC,k} \\ w_{RC,k} \end{bmatrix}$

The expectation of a random vector is defined component-wise,

$$\mathbb{E}[\mathbf{w}] = \begin{bmatrix} \mathbb{E}[w_1] \\ \mathbb{E}[w_2] \\ \vdots \\ \mathbb{E}[w_n] \end{bmatrix}$$

Vector Zero-Mean White Gaussian Noise (2/3)

The sequence $\mathbf{w}_k$ is called vector zero-mean white Gaussian noise if it satisfies:

- **Zero Mean** $E[\mathbf{w}_k] = 0$, meaning

$$E[\mathbf{w}_k] = \begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix}$$

No systematic bias in any state direction (expected value of the random vector is a vector whose components are the expected values of each individual element).

- **Gaussian Probability Density Function (PDF)**

$$p(\mathbf{w}_k) = \frac{1}{\sqrt{(2\pi)^n \det(Q)}} \exp\left(-\frac{1}{2} \mathbf{w}_k^T Q^{-1} \mathbf{w}_k\right) \text{ or } \mathbf{w}_k \sim \mathcal{N}(0, Q) \text{ (compact notation)}$$

Each vector sample follows a **multivariate normal distribution**:

where:

- $Q \in \mathbb{R}^{n \times n}$ = covariance matrix,
- $\det(Q)$ = determinant of Q .

Vector Zero-Mean White Gaussian Noise (3/3)

- **White Property (No Time Correlation)**

$$E[\mathbf{w}_k \mathbf{w}_j^T] = \begin{cases} Q, & k = j \\ 0, & k \neq j \end{cases}$$

- noise at different times is independent (therefore uncorrelated),
- every timestep generates a new random value, injecting new uncertainty.

Covariance Matrix Q

$$Q = E[\mathbf{w}_k \mathbf{w}_k^T] = \begin{bmatrix} \sigma_1^2 & \text{cov}_{12} & \cdots \\ \text{cov}_{21} & \sigma_2^2 & \cdots \\ \vdots & & \ddots \end{bmatrix}$$

- **Diagonal terms** σ_i^2 are the variance of disturbance affecting each state.
- **Off-diagonal terms:** Correlation between disturbances: $E[w_{i,k} w_{j,k}]$

The covariance matrix of a random vector contains:

- the variances of each component on the diagonal and
- the covariances between components off the diagonal.

White Gaussian noise zero mean

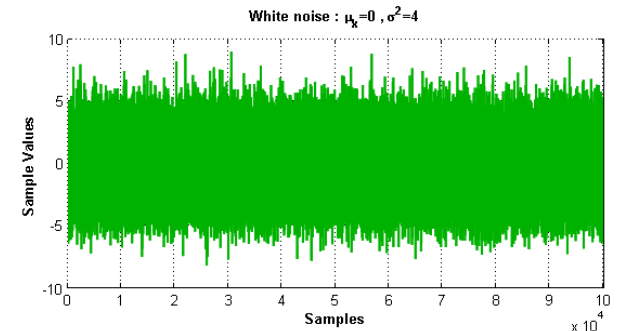
In Kalman filtering, **zero-mean white Gaussian noise** is used to model:

- **process noise** w_k
- **measurement noise** v_k
- $E[w_k] = 0, E[v_k] = 0 \rightarrow$ the noise introduces no bias (fluctuates around zero)
- $w_k \sim \mathcal{N}(0, Q), v_k \sim \mathcal{N}(0, R)$

Q — process noise covariance

R — measurement noise covariance

These matrices define the **variance (uncertainty level)** of the noise, i.e. noise intensity



On Q and R

- Usually, in Kalman filter implementations, the covariance matrices Q and R are assumed constant, meaning that the statistical properties of the noise do not change over time.
- However, the actual realizations of the process and measurement noise are not constant. The noise values vary randomly at every time step.

NOTE: Formally, w_k and v_k are **random sequences indexed by time**

$$w_k \sim \mathcal{N}(0, Q)$$

$$v_k \sim \mathcal{N}(0, R)$$

Even if Q and R are constant,

$$w_0, w_1, w_2, \dots$$

are different random vectors at each time step.

What stays constant is only the distribution from which they are drawn.

On Q and R

- If the measurement is scalar, the measurement noise covariance R is also a scalar.
- It represents the variance of the measurement noise, which describes the spread (uncertainty) of the sensor error.

$$R = \sigma_v^2$$

- In contrast, the process noise covariance $Q \in \mathbb{R}^{n \times n}$ is a matrix, since the process noise affects multiple states.
- In many applications, Q is assumed diagonal, meaning the disturbances affecting different states are uncorrelated.

Process noise vector and Q

Example: Suppose the process noise is

$$w_k = \begin{bmatrix} w_{1,k} \\ w_{2,k} \\ w_{3,k} \end{bmatrix}$$

- Each component represents a disturbance affecting one state.
- **Covariance matrix** $Q = E[w_k w_k^T]$

Expanded:

$$Q = \begin{bmatrix} E[w_{1,k}^2] & E[w_{1,k}w_{2,k}] & E[w_{1,k}w_{3,k}] \\ E[w_{2,k}w_{1,k}] & E[w_{2,k}^2] & E[w_{2,k}w_{3,k}] \\ E[w_{3,k}w_{1,k}] & E[w_{3,k}w_{2,k}] & E[w_{3,k}^2] \end{bmatrix}$$

If disturbances are uncorrelated, i.e. $E[w_{i,k}w_{j,k}] = 0 \ i \neq j$

So, the covariance matrix becomes **diagonal**

$$Q = \begin{bmatrix} \sigma_1^2 & 0 & 0 \\ 0 & \sigma_2^2 & 0 \\ 0 & 0 & \sigma_3^2 \end{bmatrix} \quad \begin{array}{l} \sigma_1^2 \rightarrow \text{variance of disturbance affecting state 1} \\ \sigma_2^2 \rightarrow \text{variance of disturbance affecting state 2} \\ \sigma_3^2 \rightarrow \text{variance of disturbance affecting state 3} \end{array}$$

- State-space model (Kalman filter)

$$\begin{aligned}x_{k+1} &= Ax_k + Bu_k + \mathbf{w}_k \\y_k &= Cx_k + \mathbf{v}_k\end{aligned}$$

$w_k \sim \mathcal{N}(0, Q)$ $w_k \in \mathbb{R}^n \longrightarrow Q \in \mathbb{R}^{n \times n}$ is the covariance matrix.

$$w_k = \begin{bmatrix} w_k^{(1)} \\ w_k^{(2)} \\ \vdots \\ w_k^{(n)} \end{bmatrix}$$

$v_k \sim \mathcal{N}(0, R)$ $v_k \in \mathbb{R} \longrightarrow R$ is a scalar variance.

- v and w are independent variables $\rightarrow$ the noise on the measurement (sensor) does not depend on the uncertainty on the process (or process noise)
- The initial state $x(0)$ is a random vector with given mean and covariance. The initial state is assumed to be statistically independent of the process noise w and the measurement noise v .
- The state $x(k)$ and output $y(k)$ are now gaussian noise vectors because they are linear combination of gaussian noise vectors
- The main idea is to use the knowledge of the model (A,B,C,D) and the probabilistic description of the noise (process and measurement) to clean up the measurement and estimate the state
- The output of the Kalman filter is the **estimated state vector** $\hat{x}$, which represents the **best probabilistic estimate of the true system state** given the model and the noisy measurements.

Kalman filter

- The Kalman filter is a recursive optimal estimator that computes the estimate of the system state by minimizing the expected value of the squared estimation error (mean-square error) based on noisy measurements.
- The Kalman filter estimate is the state that **minimizes the expected squared estimation error given the measurements:**

$$\hat{x}_k = \arg \min_{\hat{x}} \mathbb{E}[(x_k - \hat{x})^T (x_k - \hat{x}) \mid y_{1:k}]$$

- x_k : true (unknown) state
- $\hat{x}_k$: estimated state
- $y_{1:k}$: all measurements up to time k
- $\mathbb{E}[\cdot \mid y_{1:k}]$: expectation **conditioned on the available measurements**

Thus, the Kalman filter minimizes the expected squared estimation error, also called the mean-square error (MSE).

Notation

Variable	Alternative symbol	Meaning
$\hat{x}_{k k-1}$	$\hat{x}_k^-$	the a priori state estimate at step k given the knowledge of the state prior to step k – using only the process equation (model, no measurement is used here)
$\hat{x}_{k k}$	$\hat{x}_k$	the a posteriori state estimate at step k using the a priori state estimate and measurement
$e_{k k-1} = x_k - \hat{x}_{k k-1}$	$e_k^- = x_k - \hat{x}_k^-$	the a priori estimate error
$e_{k k} = x_k - \hat{x}_{k k}$	$e_k = x_k - \hat{x}_k$	the a posteriori estimate error
$P_{k k-1} = E[e_{k k-1}e_{k k-1}^T]$	$P_k^- = E[e_k^-e_k^{-T}]$	the a priori estimate error covariance
$P_{k k} = E[e_{k k}e_{k k}^T]$	$P_k = E[e_k e_k^T]$	the a posteriori estimate error covariance

Predictor-corrector scheme

- The Kalman Filter provides not only the best estimate in the mean squared error sense, but also a measure of how confident that estimate is. This confidence is reflected in the covariance matrix P , which decreases as the filter converges and gains more information.
- Structure of a linear recursive filter written in a predictor-corrector format

Predictor-corrector scheme

Prediction Step

A priori estimate: we forecast how the state has evolved only on the basis of the model: $\hat{x}_{k|k-1}$

→ we propagate the **estimated state**,

$$\hat{x}_{k|k-1} = A\hat{x}_{k-1|k-1} + Bu_{k-1}$$

→ Predicted measurement:

$$\hat{y}_{k|k-1} = C\hat{x}_{k|k-1}$$

Correction Step

A posteriori estimate: at time step k , once the measurement y_k is available, the state estimate is updated by correcting the predicted state using the innovation weighted by the Kalman gain:

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k(y_k - C\hat{x}_{k|k-1})$$

where

$y_k - C\hat{x}_{k|k-1}$ is the innovation (measurement residual), i.e., the difference between the actual measurement and the predicted measurement.

Kalman Filter Design

The **predictor step** uses information from the state equation

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + \mathbf{K}(y_k - \hat{y}_{k|k-1})$$

The **corrector step** uses new measurement available at time k

The Kalman Gain $\mathbf{K}$ (vector $n \times 1$) is chosen to minimize the *a posteriori* error covariance

$$e_{k|k} = x_k - \hat{x}_{k|k-1}$$

$$P_{k|k} = E[e_{k|k} e_{k|k}^T]$$

Prediction step

Prediction of the states

$$\hat{x}_{k|k-1} = A \hat{x}_{k-1} + B u_{k-1}$$

Covariance of the prediction error

$$P_{k|k-1} = A P_{k-1|k-1} A^T + Q$$



Initialization

$$P(0), \hat{x}(0), Q, R$$

Correction step

Calculation of the correction gain (Kalman Gain)

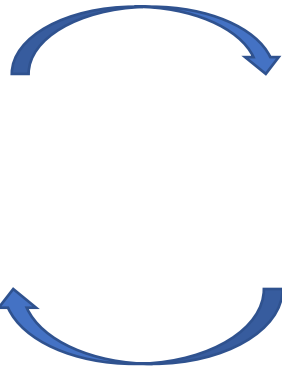
$$K_k = P_{k|k-1} C^T (C P_{k|k-1} C^T + R)^{-1}$$

Correction of the estimation with the measurements (Current estimate)

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k (y_k - \hat{y}_{k|k-1})$$

Update the error in the estimate

$$P_{k|k} = (I - K_k C) P_{k|k-1}$$



$P_{k|k-1}$ = the uncertainty in the predicted state before incorporating the measurement y_k .

Estimation error covariance matrix

$P(0)$ (or $P_{0|0}$) represents the initial estimation error covariance matrix:

$$P_{0|0} = \mathbb{E}[(x_0 - \hat{x}_{0|0})(x_0 - \hat{x}_{0|0})^T]$$

where

- x_0 = true initial state
- $\hat{x}_{0|0}$ = initial estimate of the state
- $P_{0|0}$ = covariance of the **initial estimation error**

$P(0)$ quantifies how uncertain we are about the initial state estimate.

- **Large** $P(0)$ $\rightarrow$ high uncertainty about the initial state
- **Small** $P(0)$ $\rightarrow$ strong confidence in the initial estimate

Kalman Gain

- What we do in the calculation of the Kalman gain is to put relative importance in the error in the estimate versus the error in the data!
- If the **error in the estimate** is **small**, we put more emphasis in it, if the **error in the data** is **small** we put more emphasis in it (we trust the data more)

Kalman Gain

Error in the estimate $\rightarrow E_{EST}$ Error in the data/measurement $\rightarrow E_{MEA}$

$$K_k = P_{k|k-1} C^T (C P_{k|k-1} C^T + R)^{-1} \quad \text{Conceptually } \Rightarrow \frac{E_{EST}}{E_{EST} + E_{MEA}}$$

Error in the estimate $\rightarrow E_{EST}$

The Kalman gain K_k determines how much the estimator trusts the new measurement compared to the previous estimate:

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k (y_k - C \hat{x}_{k|k-1})$$

- Current (at time k) Estimate EST_k
- Previous (at time $k-1$) estimate EST_{k-1}
- Measurements MEA

$$EST_k = EST_{k-1} + K_k [MEA - EST_{k-1}]$$

On Kalman Gain

1. If K_k is large $\rightarrow$ the measurements are relatively accurate, meaning the measurement noise covariance R is small, and the measurement error E_{MEA} is small
 - In this case, we want the estimate to follow the measurement closely. $MEA-EST_{k-1}$ is multiplied by a large (K_k) number:
 - The estimator puts strong weight on the measurement
 - The estimate moves quickly toward the measured value

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k(y_k - C\hat{x}_{k|k-1})$$

$\rightarrow$ sensor is reliable, we trust the measurement.

On Kalman Gain

- If K_k is small it means that the measurement is relatively unreliable compared to the current estimate $\rightarrow$ **measurement noise covariance** R is large, meaning the sensor readings contain significant noise or uncertainty.
- In this situation, the measurement error E_{MEA} is large relative to the estimation error. Therefore, the Kalman filter assumes that the new measurement may not provide trustworthy information about the true state.

the **innovation** $y_k - C\hat{x}_{k|k-1}$ is multiplied by a **small Kalman gain**.

- Because the gain is small, the correction applied to the estimate is also **small** $\rightarrow$ We don't want the new measurement to "upset" the estimate

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k(y_k - C\hat{x}_{k|k-1})$$

The estimator **relies primarily on the model prediction** rather than the measurement $\hat{x}_k \approx \hat{x}_{k|k-1}$

Evolution of the Kalman Gain Over Time

- As the Kalman filter processes more measurements, the uncertainty in the state estimate decreases. This uncertainty is represented by the state estimation covariance matrix P_k .
- Because the estimator becomes more confident about the state, the Kalman gain tends to decrease over time.
- This means that:
 - In the early stages, when the estimate is still uncertain, the Kalman gain is relatively large, and the filter relies more heavily on measurements to correct the estimate.
 - As more measurements are incorporated, the estimator learns the system behavior, the uncertainty decreases, and the Kalman gain becomes smaller.

Summary

- Kalman gain matrix $\mathbf{K}(k)$ varies with time.
- For a LTI system then after an initial interval of time during which $\mathbf{K}_k$ does not vary with time, $\mathbf{K}_k$ reaches a steady state in which all its elements are constants.

- Luenberger observer

$$\hat{x}_{k+1} = A\hat{x}_k + Bu_k + L(y_k - C\hat{x}_k)$$

- Kalman filter

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k(y_k - C\hat{x}_{k|k-1})$$

Key difference:

- **Luenberger:** gain L chosen by pole placement
- **Kalman:** gain K_k computed **optimally from noise statistics**

Extended Kalman Filter (EKF)

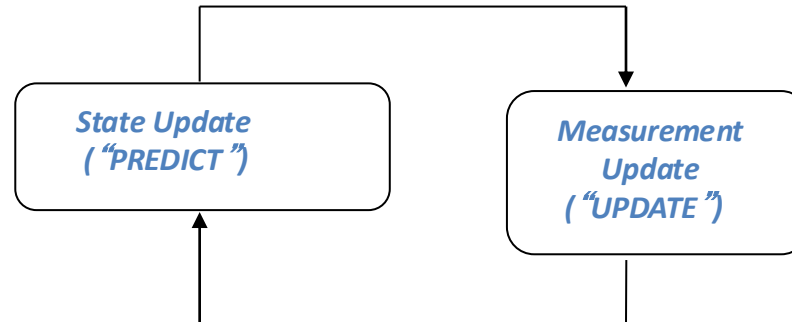
EKF

- Many practical systems have non-linear state update or nonlinear measurement equations.
- Kalman filter can be applied to a linearized version of these equations with loss of optimality.
- The EKF linearizes the state and measurement equations about the predicted state as an operating point.
- Discrete time non linear dynamic system with additive white noise that models unpredictable disturbances.

$$\begin{aligned}x_k &= f(x_{k-1}, u_{k-1}) + w_k \\y_k &= h(x_k) + v_k\end{aligned}$$

EKF

- Unlike its linear counterpart, the extended Kalman filter in general is *not* an optimal estimator
- In addition, if the initial estimate of the state is wrong, or if the process is modeled incorrectly, the filter may diverge.
- The basic operations (prediction and correction steps) of the EKF are the same as the linear discrete Kalman filter.



Prediction step

Prediction of the states

$$\hat{x}_{k|k-1} = f(x_{k-1}, u_{k-1})$$

Covariance of the prediction error

$$P_{k|k-1} = F_k P_{k-1|k-1} F_k^T + Q$$

$$F_k = \left. \frac{\partial f}{\partial x} \right|_{\hat{x}_{k|k-1}, u_k}$$

$$H_k = \left. \frac{\partial h}{\partial x} \right|_{\hat{x}_{k|k-1}}$$

Linearization

Correction step

Calculation of the correction gain (Kalman Gain)

$$K_k = P_{k|k-1} H_k^T (H_k P_{k|k-1} H_k^T + R)^{-1}$$

Correction of the estimation with the measurements (Current estimate)

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k (y_{meas,k} - \hat{y}_k)$$

Update the error in the estimate

$$P_{k|k} = (I - K_k H_k) P_{k|k-1}$$

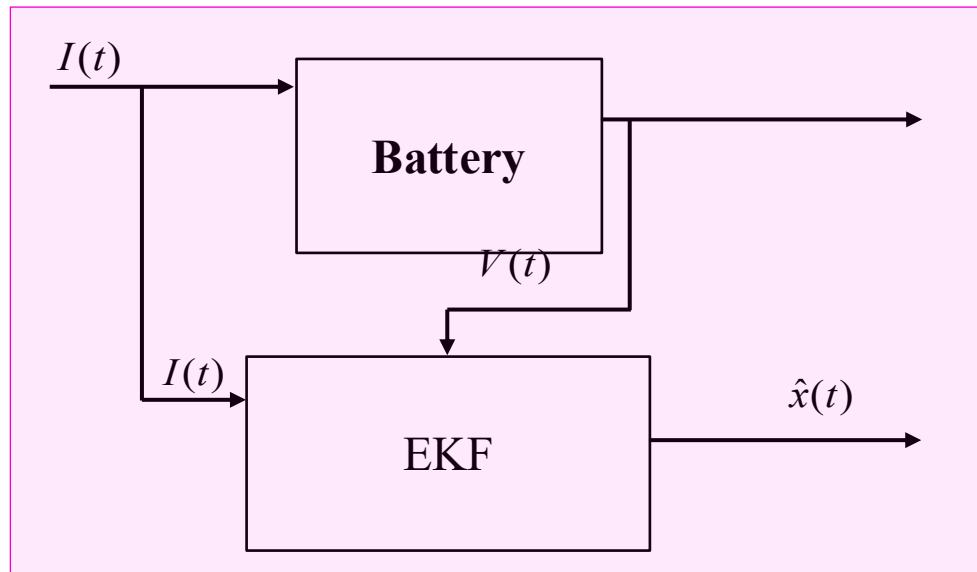
Initialization

$$P(0), \hat{x}(0), Q, R$$

EKF- Lithium Ion Battery

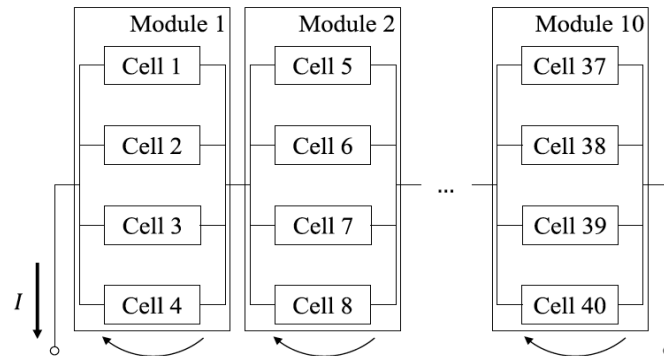


Taborelli, C., Onori, S., Maes, S., Sveum, P., Al-Hallaj, S., Al-Khayat, N., "Advanced battery management system design for SOC/SOH estimation for e-bikes applications", *International Journal of Powertrains (IJPT)*, vol. 5, n. 4, 2016 <http://dx.doi.org/10.1504/IJPT.2016.081795>



Battery System

- The battery pack is made by 40 cells. The cells have a nominal voltage of 3.7 V and a rated capacity of 2.6 Ah. In order to achieve the voltage and capacity requirements, the battery pack is built with the structure 10s4p



36V 10AH

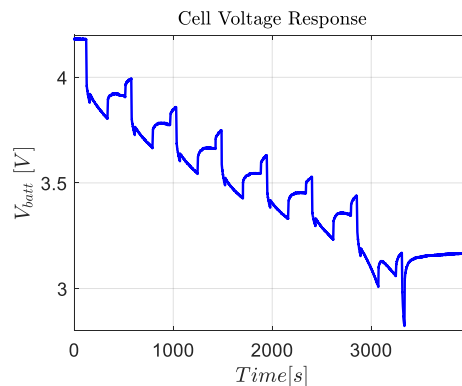
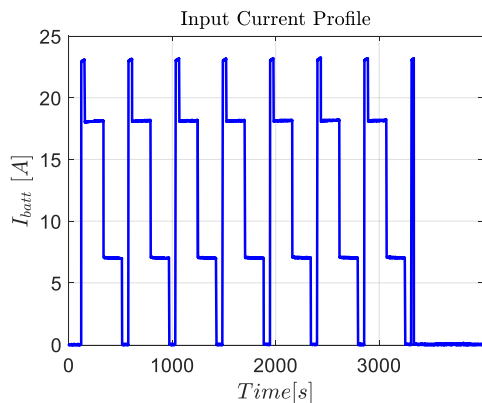
NMC - Charging time 4 hours (empty to full)

Life Cycle: 700-2000 charges (depending on use)

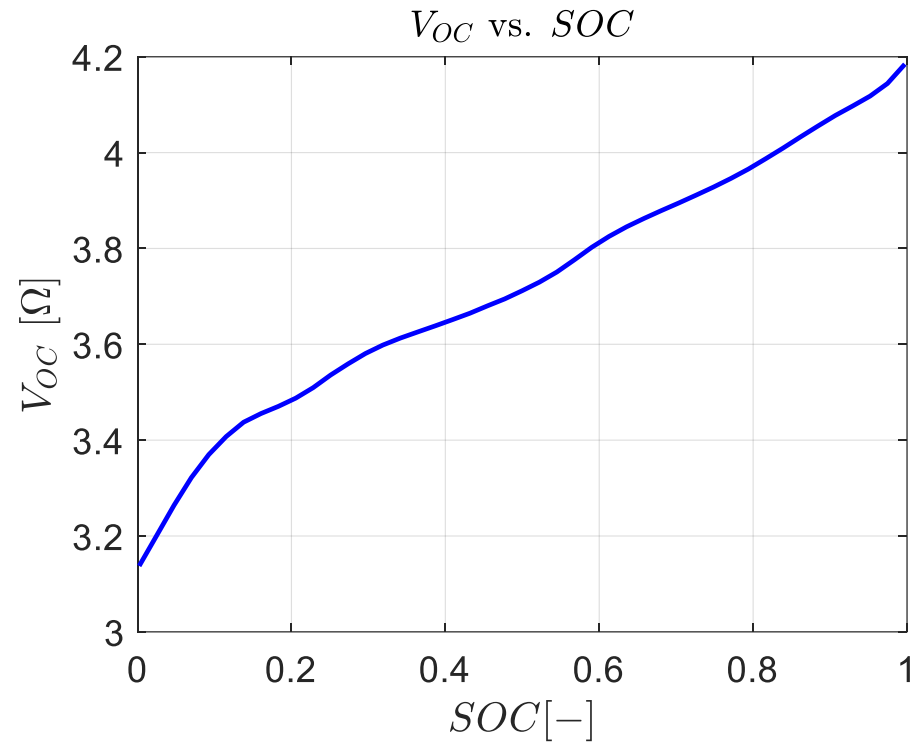
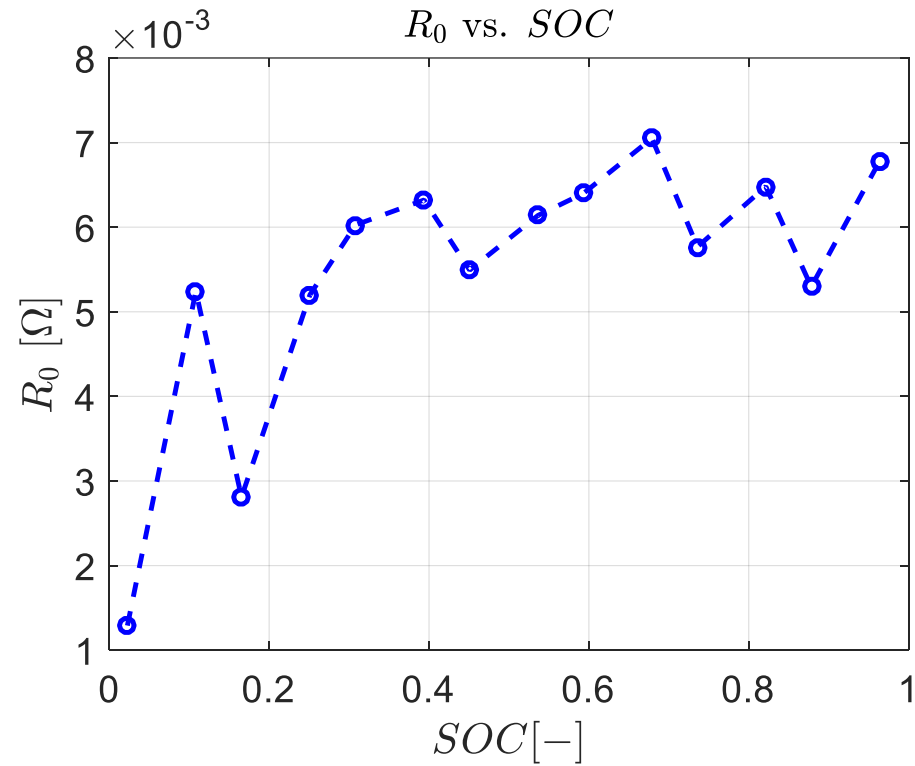
BMS inside pack

Weight 5.5 Lbs (2.77kg)

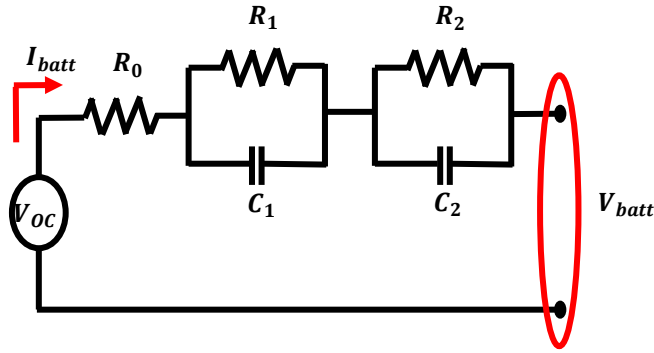
Size: 9 x 3 x 4 inches (228 x 74 x 98 mm)



Data → System nonlinearities



2nd order ECM



$$\left. \begin{aligned} \frac{dSOC}{dt} &= -\frac{I_{batt}}{Q_n} \\ \frac{dV_1}{dt} &= -\frac{1}{R_1 C_1} V_1 + \frac{1}{C_1} I_{batt} \\ \frac{dV_2}{dt} &= -\frac{1}{R_2 C_2} V_2 + \frac{1}{C_2} I_{batt} \end{aligned} \right\} \dot{x}(t) = Ax(t) + Bu(t)$$

$$V_{batt}(t) = V_{OC}(SOC) - R_0(SOC)I_{batt}(t) - V_1(t) - V_2(t)$$

We assume that only V_{OC} and R_0 are a function of SOC $\rightarrow$ nonlinearity only in the output equation

$$\dot{x}(t) = \begin{bmatrix} 0 & 0 & 0 \\ 0 & \frac{-1}{\tau_1} & 0 \\ 0 & 0 & \frac{-1}{\tau_2} \end{bmatrix} x(t) + \begin{bmatrix} \frac{-1}{Q} \\ \frac{1}{C_1} \\ \frac{1}{C_2} \end{bmatrix} u(t)$$

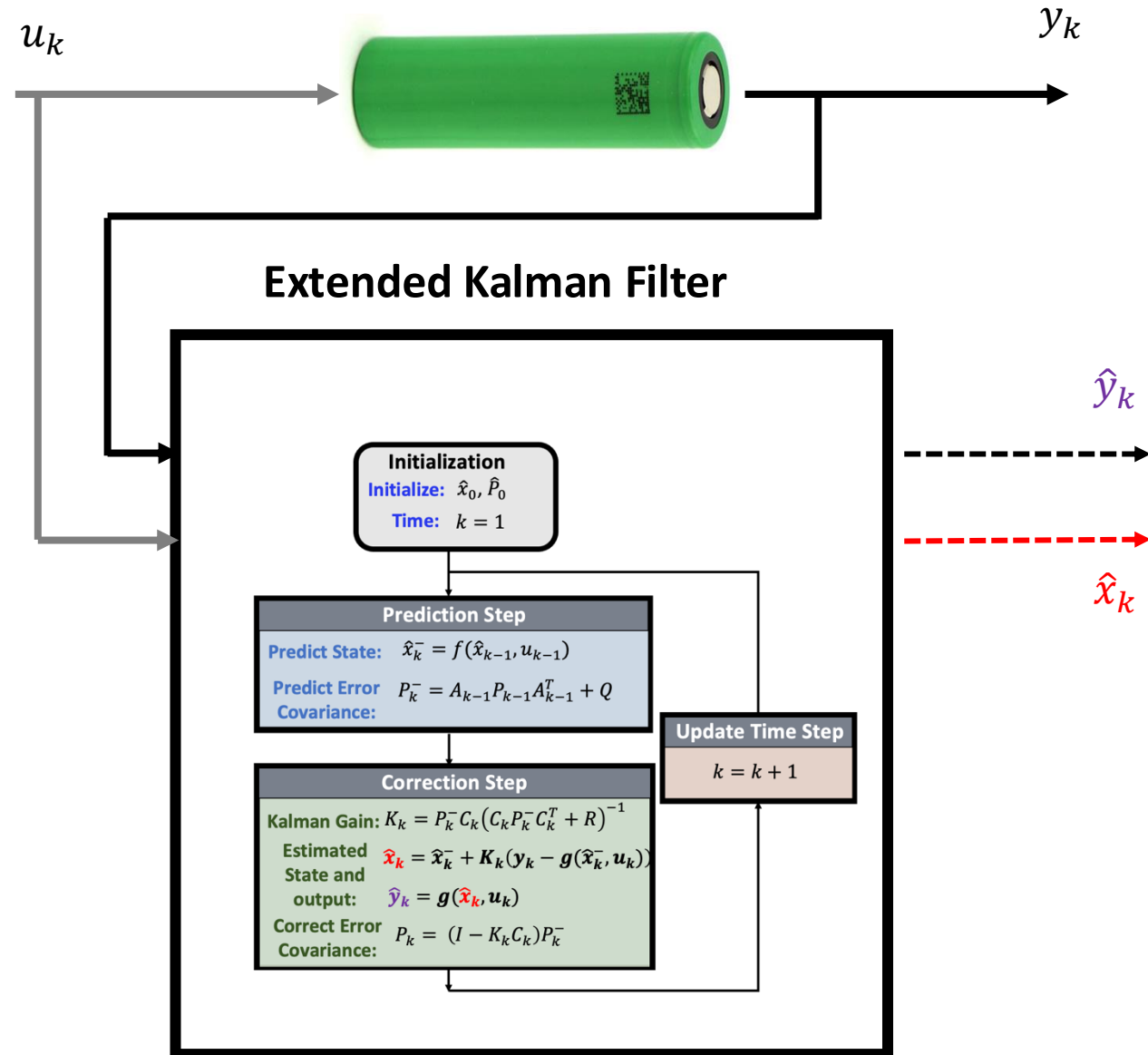
$$y(t) = g(x, u)$$

$$\begin{aligned} \tau_1 &= R_1 C_1 \\ \tau_2 &= R_2 C_2 \end{aligned}$$

EKF

Linearization of the 2nd order ECM will involve only the output equation

$$C = \left. \frac{\partial h}{\partial x} \right|_{x_0}$$



Initialization

- The covariance matrix Q of the process noise is designed under the assumption that there is no correlation between the noise among state components leading to a diagonal structure.
- For example, an initial choice might be

$$Q = \begin{bmatrix} 1000 \cdot R & 0 & 0 \\ 0 & 0.1 \cdot R & 0 \\ 0 & 0 & 0.01 \cdot R \end{bmatrix}$$

with R the sensor noise covariance, $R = 8.432\text{e-}4$

- Initial Error Covariance Matrix

$$P_0 = \begin{bmatrix} 0.1 & 0 & 0; \\ 0 & 0.001 & 0; \\ 0 & 0 & 0.001 \end{bmatrix};$$

Initialization

```
% EKF Initialization  
P_t_init = [.1 0 0;0 0.001 0;0 0 0.001]; % initial covariance  
x_t_init=[0; 0; 0]; %initial state  
x_temp=[0;0;0]; % temporary variable for calculation  
R = 8.432e-4; % measurement noise covariance  
Q = diag([1000*R .1*R .01*R]); % process noise covariance  
delta_soc = 0.01; % delta soc for linearization
```

Jacobian Matrices (Linearization)

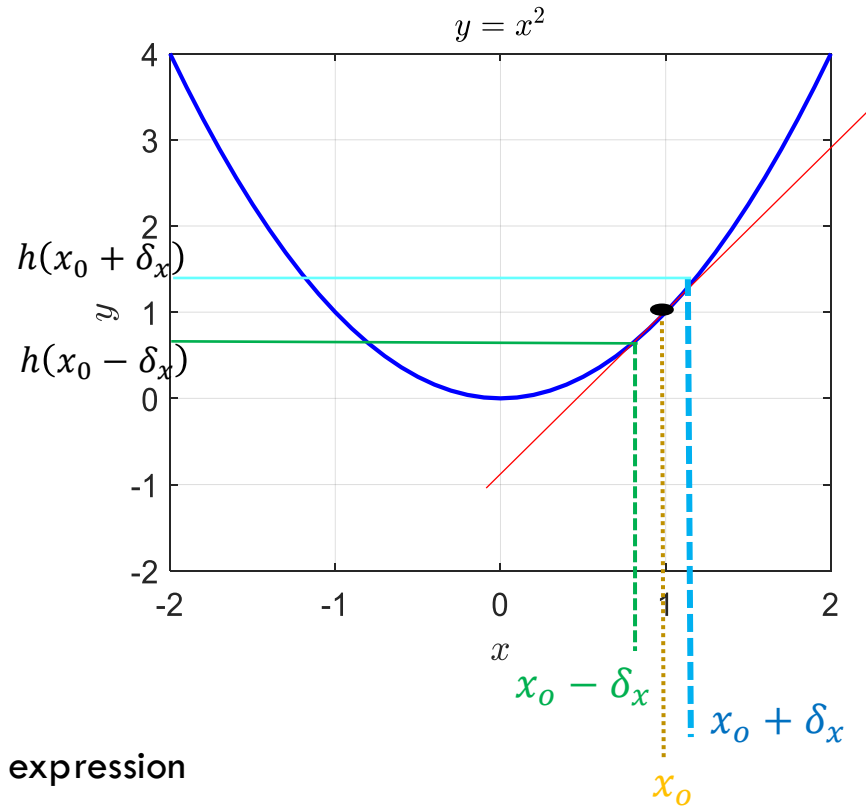
Linearization Steps:

1. Perturb around the operating point by δ_x
2. Ensure δ_x is very small (assume $\delta_x = 0.01$)

3. The Jacobian is: $\left. \frac{\partial h}{\partial x} \right|_{x=x_0}$

$$\begin{aligned} \left. \frac{\partial h}{\partial x} \right|_{x_0} &= \frac{h(x_0 + \delta_x) - h(x_0 - \delta_x)}{(x_0 + \delta_x) - (x_0 - \delta_x)} \\ &= \frac{h(x_0 + \delta_x) - h(x_0 - \delta_x)}{2\delta_x} \end{aligned}$$

4. Evaluate $h(x_0 + \delta_x)$ and $h(x_0 - \delta_x)$ in the above expression from the given nonlinearity $y = h(x)$ function.



Matlab code

```
%At each time step

% Linearization via jacobian around the estimated state

Voc_jb_1(i) = interp1(SOC1d_lg,OCV1d_lg,x_t_init(1,1)-delta_soc,'linear','extrap');
Voc_jb_2(i) = interp1(SOC1d_lg,OCV1d_lg,x_t_init(1,1)+delta_soc,'linear','extrap');

R0_jb_1(i) = interp1(socR_values_const,R_values_const,x_t_init(1,1)-delta_soc,'linear','extrap');
R0_jb_2(i)=interp1(socR_values_const,R_values_const,x_t_init(1,1)+delta_soc,'linear','extrap');

jacobian_Voc(i) = (Voc_jb_2(i)-Voc_jb_1(i))/(2*delta_soc);
jacobian_R0(i) = (R0_jb_2(i)-R0_jb_1(i))/(2*delta_soc);
jacobian_Voc_R0(i) = jacobian_Voc(i)-jacobian_R0(i)*I_data(i);
```

Linearized Dynamics

```
.....  
% Linearized State Space Model  
A = [0, 0, 0; 0, -1/(R1*C1), 0; 0, 0, -1/(R2*C2)];  
B = [-1/capacity; 1/C1; 1/C2];  
C = [jacobian Voc R0(i), -1, -1];  
D = [0];
```

$$\dot{x}(t) = \begin{bmatrix} 0 & 0 & 0 \\ 0 & \frac{-1}{\tau_1} & 0 \\ 0 & 0 & \frac{-1}{\tau_2} \end{bmatrix} x(t) + \begin{bmatrix} \frac{-1}{Q} \\ \frac{1}{C_1} \\ \frac{1}{C_2} \end{bmatrix} u(t)$$

$$y(t) = \left[\left(\frac{\partial V_{oc}}{\partial SOC} - \frac{\partial R_0}{\partial SOC} \right) \quad -1 \quad -1 \right] x(t)$$

For the purpose of implementing EKF, only the A,B,C matrices of the linearized system are needed

From Continuous-time to Discrete-Time Dynamics

$$\begin{cases} SoC(k+1) = SoC(k) - \frac{\Delta t}{Q_{nom}} I(k) \\ V_1(k+1) = e^{-\frac{\Delta t}{\tau_1}} V_1(k) + R_1 \left(1 - e^{-\frac{\Delta t}{\tau_1}}\right) I(k) \\ V_2(k+1) = e^{-\frac{\Delta t}{\tau_2}} V_2(k) + R_2 \left(1 - e^{-\frac{\Delta t}{\tau_2}}\right) I(k) \end{cases}$$



$$x(k+1) = Ax(k) + Bu(k) \quad A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & e^{-\frac{\Delta t}{\tau_1}} & 0 \\ 0 & 0 & e^{-\frac{\Delta t}{\tau_2}} \end{bmatrix} \quad B = \begin{bmatrix} -\frac{\Delta t}{Q_{nom}} \\ R_1 \left(1 - e^{-\frac{\Delta t}{\tau_1}}\right) \\ R_2 \left(1 - e^{-\frac{\Delta t}{\tau_2}}\right) \end{bmatrix}$$

$$y(k) = \left[\left(\frac{\partial V_{oc}}{\partial SOC} - \frac{\partial R_0}{\partial SOC} \right) \quad -1 \quad -1 \right] x(k)$$

```
% Check observability
Ob = obsv(A,C);
```

```
%sampling time
dt=0.1
```

```
% Discrete state-space Model
CT_ss=ss(A,B,C,D);
DT_ss=c2d(CT_ss,dt);
A_discrete = DT_ss.a;
B_discrete = DT_ss.b;
C_discrete = DT_ss.c;
D_discrete = DT_ss.d;
```

EKF - predictor

`% Predictor (A-priori)`

Update the state using the discrete equation

`P_tp = A_discrete * P_t_init * A_discrete' + Q;`

`% Error covariance of the a priori estimate`

EKF – Correction/Update

```
% Innovation Calculation
```

```
Inn(i) = V_expt(i) - (Nonlinear output model) % innovation
```

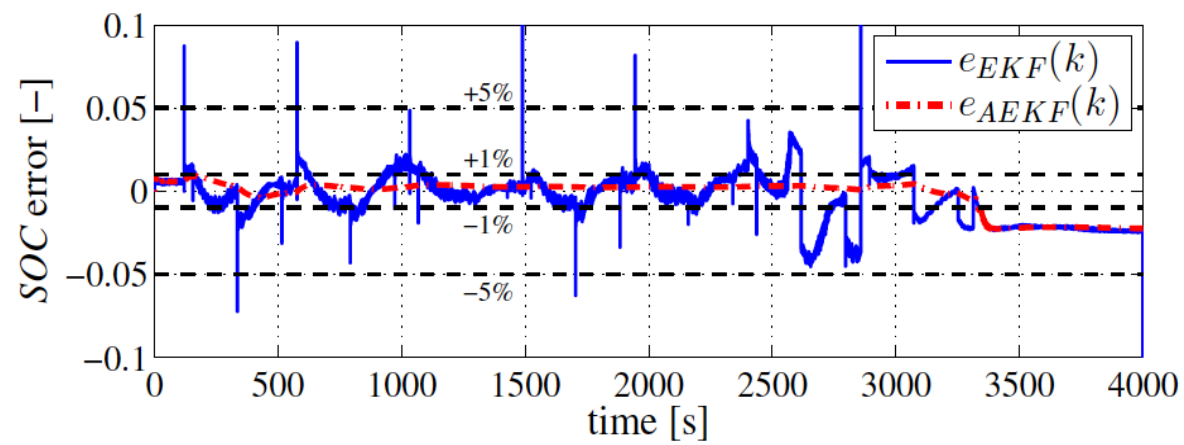
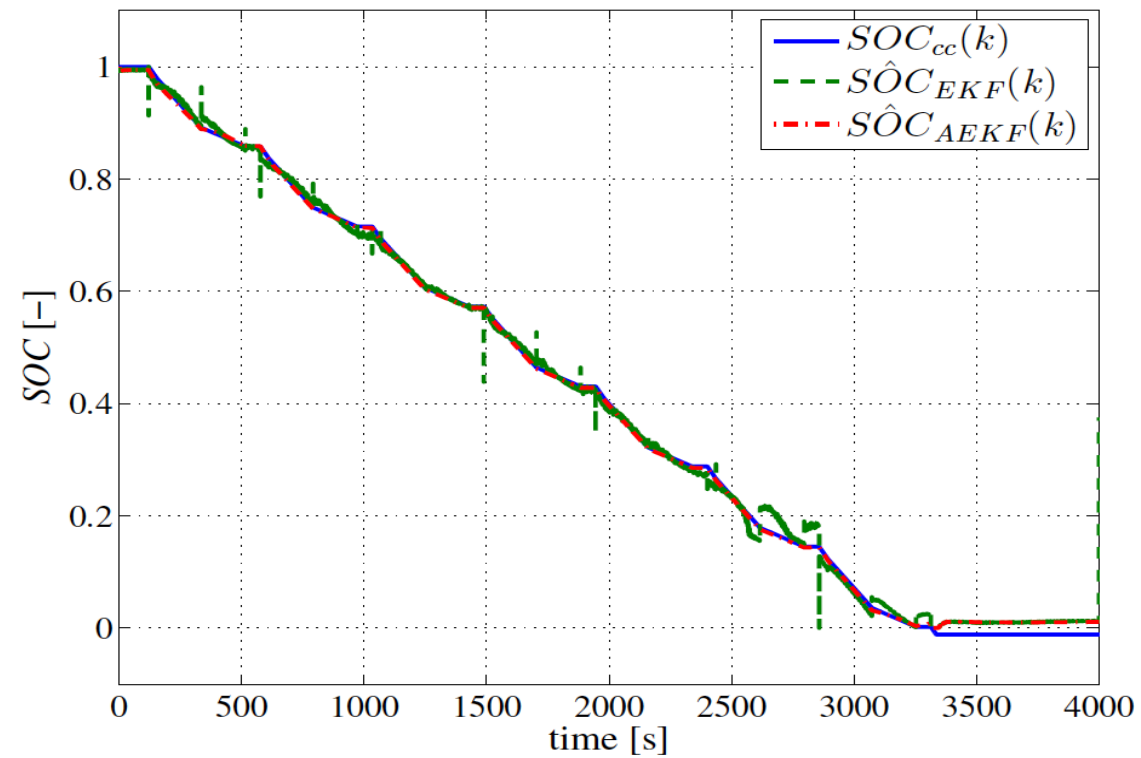
```
S = C_discrete * P_tp * C_discrete' + R; %Innovation covariance
```

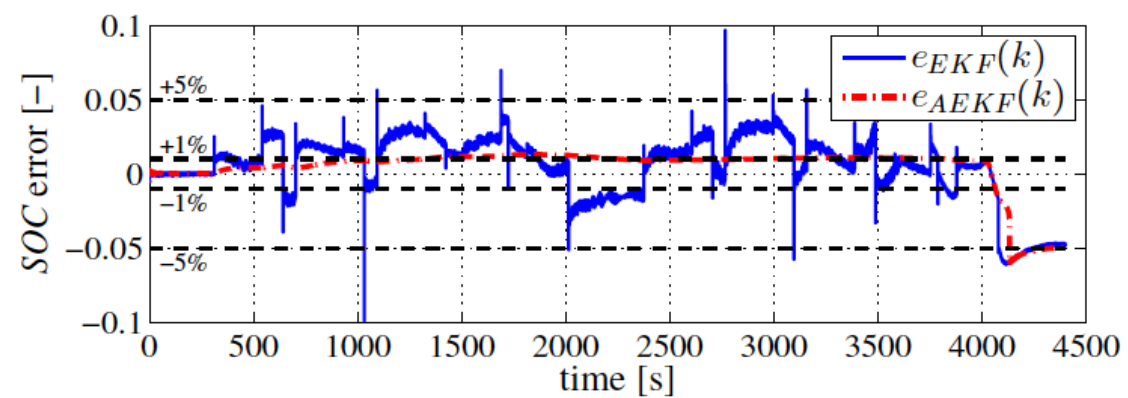
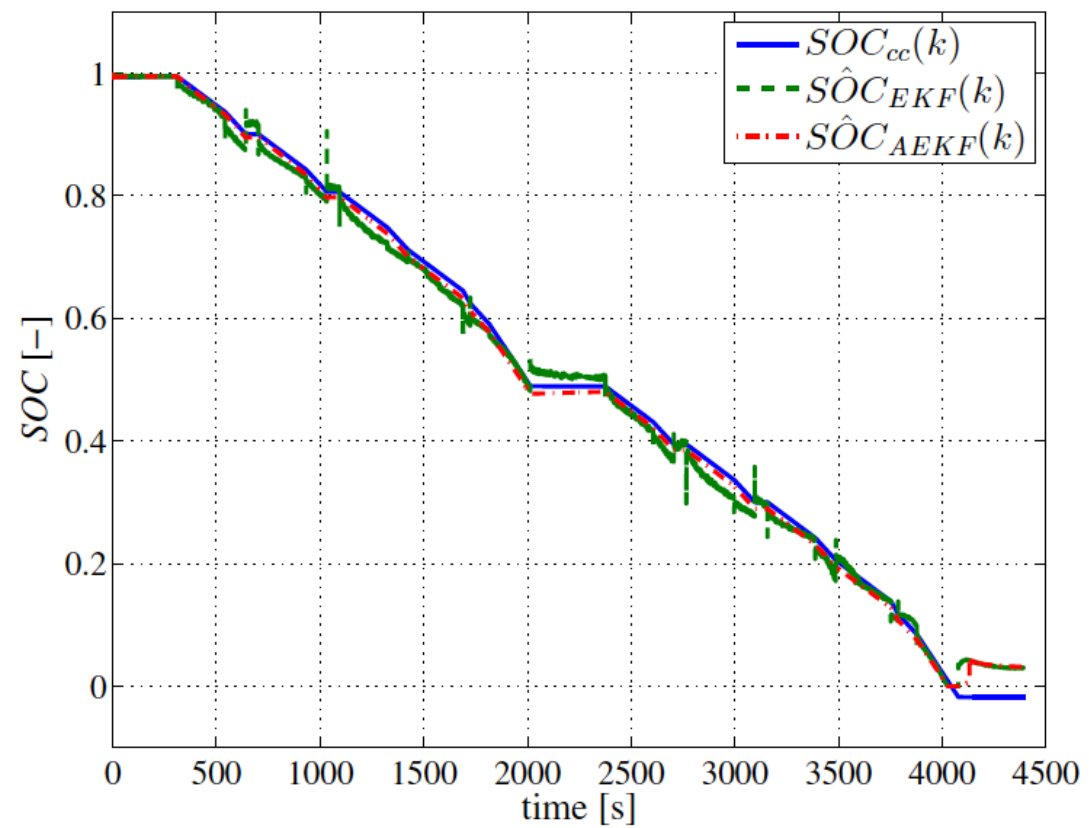
```
% Corrector (A-posteriori)
```

```
L = P_tp * C_discrete' / S; % kalman gain
```

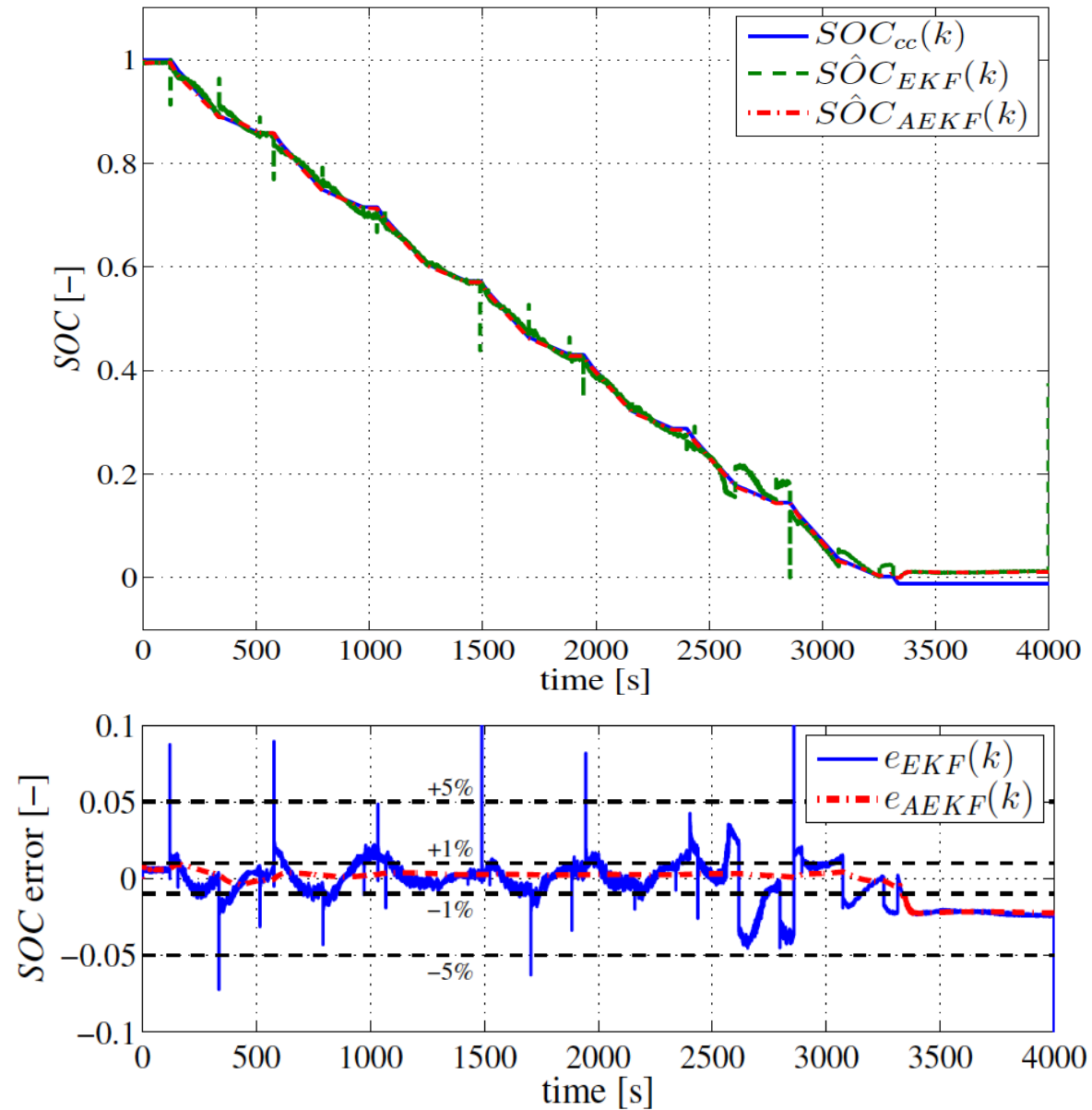
```
% update corrected state
```

```
P_kc = (eye(length(A_discrete)) - L * C_discrete) * P_tp; % update covariance
```





SOC estimation



SOC estimation

